Formal Mathematical Representation Discovery

A Research Thesis for AI-Native Mathematics

Abstract

This thesis proposes *formal mathematical representation discovery*: a research program that treats Lean/mathlib as a machine-readable record of mathematical abstraction, and builds AI systems that mine proof-state transition graphs for reusable motifs, retrieve structurally analogous theorems across domains, and detect missing abstractions through graph-theoretic bottlenecks. The central claim is that the next major advance in AI for mathematics will come not from better tactic prediction but from learning how representations are discovered, compressed, and transferred—a process that formal libraries now expose empirically for the first time.

1. Central Thesis

Modern formal mathematics libraries such as Lean/mathlib (The mathlib Community, 2020) are not merely repositories of theorems and proofs. They are large-scale, machine-readable records of how mathematical knowledge is represented, compressed, transported, and reused. Their internal structures—definitions, typeclasses, proof states, tactic traces, dependency graphs, coercion paths, simplification rules, and theorem statements—encode a computational shadow of mathematical abstraction itself.

The central thesis of this research program is:

> The next major advance in AI for mathematics will not come only from better next-tactic prediction, larger language models, or brute-force proof search. It will come from learning how mathematical representations are discovered, reused, compressed, and transferred across domains.

A sharper formulation of the same idea:

> Proof is not just search in a fixed graph. Advanced proof is search over representations that reshape the graph. Lean/mathlib gives us the first large-scale substrate for studying this process computationally.

Most current AI theorem proving systems operate at one level:

```
proof state → retrieve premises → generate tactic → search
```

This research operates one level higher:

```
formal library / proof traces → detect motifs, bottlenecks,
analogies, missing abstractions → reshape the search space
```

A formal mathematics system such as Lean gives us, for the first time, an empirical substrate for studying this process. It exposes the topology of proof search, the dependency geometry of mathematical concepts, the recurring motifs of proof construction, and the places where existing representations are insufficient. By mining these structures, we can build systems that retrieve analogies, suggest intermediate lemmas, detect missing abstractions, and eventually help invent new mathematical representations.

This research direction can be summarized as **formal mathematical representation discovery**.

Its goal is not simply to prove more Lean theorems. Its goal is to understand and automate the higher-level acts that make proof possible: choosing the right object, introducing the right invariant, transporting structure across domains, and compressing repeated proof patterns into reusable abstractions.

2. Motivation

Current AI theorem proving is largely framed as a local search problem. A model sees a proof state and predicts a tactic, a lemma, or a proof term. Recent systems such as AlphaProof (Google DeepMind AlphaProof Team, 2025), DeepSeek-Prover-V2 (DeepSeek-AI, 2025), and HyperTree Proof Search (Lample et al., 2022) have demonstrated that this framing, combined with large-scale data and reinforcement learning, can achieve remarkable results. Yet the framing is incomplete.

Human mathematical creativity rarely consists of local tactic selection alone. Much of it consists of *representation change*.

A difficult theorem often becomes tractable only after one introduces a new object, defines a better interface, proves an intermediate lemma, recognizes an analogy with another field, or reformulates the problem in a more natural language. In mature mathematics, these representational moves are often more important than the final proof script.

Formalized mathematics contains abundant evidence of this phenomenon. In mathlib, many definitions and structures exist not merely because they are mathematically natural in isolation, but because they make large families of theorems reusable. A typeclass hierarchy, a bundled structure, a coercion mechanism, a normal form, or a carefully shaped lemma can dramatically reduce proof complexity across hundreds of downstream results.

In ordinary computer science, we design data structures and then search on them. In

higher mathematics, the hardest step is often discovering the right data structure—the right abstraction—first. A clever proof does not merely search an existing space; it changes the topology of the search space by folding equivalent states, exposing invariants, creating bottleneck lemmas, and turning a huge unstructured search into a narrow, reusable path.

This suggests a central slogan for the entire program:

> **Mathematical abstraction is search-space compression.**

More precisely:

> A mathematical abstraction is valuable when it compresses proof search and increases theorem transportability.

Lean makes this view measurable. We can ask:

- Which structures reduce proof length and dependency depth?
- Which lemmas act as bottlenecks or bridges between domains?
- Which proof-state motifs recur across apparently unrelated areas?
- Which theorem statements have analogous relational structure despite different terminology?
- Where do proofs repeatedly suffer from the same representational friction?
- Can we detect missing abstractions before humans define them?

These questions move beyond theorem proving as local search and toward theorem proving as representation engineering.

3. Related Work

3.1 AI Theorem Proving Systems

The dominant paradigm in AI theorem proving frames proof search as a sequential decision problem. AlphaProof (Google DeepMind AlphaProof Team, 2025) combines a language model with AlphaZero-style reinforcement learning inside Lean, achieving silver-medal performance at the 2024 IMO. DeepSeek-Prover-V2 (DeepSeek-AI, 2025) uses RL-guided subgoal decomposition to achieve 88.9% pass rate on MiniF2F. HyperTree Proof Search (Lample et al., 2022) applies an online AlphaZero variant over and-or proof trees. These systems advance the state of the art in automated proof but operate entirely within the tactic-prediction paradigm this thesis moves beyond.

3.2 Premise Selection and Retrieval-Augmented Proving

Premise selection—choosing which library lemmas to supply to a prover—is the earliest form of library-aware proof automation. The hammer tradition (Paulson

and Blanchette, 2010; Blanchette et al., 2016) introduced tools like Sledgehammer for Isabelle, with premise filters based on symbol overlap (Meng and Paulson, 2009) and machine-learned classifiers (Kühlwein et al., 2013). Kaliszyk and Urban (2014) demonstrated that ML-guided premise selection at scale substantially increases automated proof coverage. LeanDojo (Yang et al., 2023) extends this to Lean 4, providing programmatic proof-state extraction and a retrieval-augmented prover (ReProver). LeanSearch (Gao et al., 2024) provides semantic search over Mathlib4 for informal natural-language queries. COPRA (Thakur et al., 2023) uses GPT-4 with retrieved lemmas and stateful backtracking.

This thesis's analogy retrieval component (§5.3) is related to premise selection but differs in aim: rather than retrieving lemmas to supply to a prover, it retrieves structurally analogous theorems as a source of proof strategy, using multi-layer graph structure rather than text or symbol overlap.

3.3 Graph-Based Representations of Formal Libraries

Two bodies of work have applied graph analysis to formal proof libraries, forming the closest technical predecessors to this thesis.

Paliwal et al. (2020) and Bansal et al. (2019) apply graph neural networks to HOL Light formula graphs for tactic and premise prediction. These systems use graph structure within individual theorem statements but do not mine cross-theorem motifs or study abstraction at the library level.

Li et al. (2026) present a network analysis of Lean 4's Mathlib itself, extracting its dependency structure into a multilayer graph with over 308,000 declarations and millions of edges across thousands of modules, and introduce graph decompositions distinguishing explicit from compiler-synthesized edges. This is the closest existing work to the graph extraction proposed in §5.1. The key difference is scope and use: Li et al. study Mathlib's network structure descriptively; this thesis proposes to use graph structure operationally for motif retrieval, analogy search, and abstraction detection.

Huch (2022) applies the same network-analysis approach to the Isabelle Archive of Formal Proofs (1.8M nodes, 2.8M edges). Like Li et al., this work examines static dependency centrality and does not pursue motif mining, analogy retrieval, or abstraction detection.

Blanchette et al. (2015) mine the AFP empirically for proof statistics: proof size, tactic usage distributions, dependency structure, and proof style evolution. This is the most direct precursor to the motif mining proposal in §5.2. The key distinction is that Blanchette et al. report descriptive statistics over tactic sequences; this thesis proposes to extract structured motifs from proof-state transition graphs and use them operationally for retrieval and abstraction suggestion.

PACT (Han et al., 2021) extracts self-supervised training tasks from Lean proof terms. LeanDojo (Yang et al., 2023) extracts proof states, premises, and tactic traces from Lean 4 at scale. The trace collection layer (§6.1) builds on and extends

this infrastructure.

3.4 Proof-Pattern Mining and Tactic Discovery

Proof-pattern mining. The most direct conceptual ancestors of this thesis are the ML4PG family of systems for Coq/SSReflect. Komendantskaya et al. (2012) introduce ML4PG, a machine learning plugin that clusters proof scripts by statistical features extracted from proof states and tactic sequences, identifying similarity across proof developments. Heras and Komendantskaya (2014) demonstrate that ML4PG can mine electronic Coq proof libraries and suggest proof strategies by analogy. This is the clearest ancestor of Hypothesis 1 and §5.2. The key distinctions from this thesis are: ML4PG uses shallow statistical features over tactic sequences rather than structured proof-state transition graphs; it operates on Coq/SSReflect at a scale far smaller than Lean/mathlib; and it does not address theorem analogy retrieval or missing abstraction detection.

Freitas and Whiteside (2014) identify named, reusable "proof patterns" for formal verification—recurring solutions to common proof obligations—through manual expert identification; this thesis proposes automated discovery from data at scale.

Tactic discovery. TacMiner (Xin et al., 2025) is the closest prior work to Project 1 (§5). It builds Tactic Dependence Graphs over Lean 4 proofs to identify reusable proof strategies across multiple proofs, discovers new tactic library entries, and refactors proofs into more modular forms—reporting 3× as many learned tactics as Peano and a 26% proof-size reduction. The distinction from this thesis is one of scope: TacMiner targets tactic library compression as an end in itself; this thesis uses motif structure as a stepping stone toward theorem analogy retrieval, missing abstraction detection, and representation discovery at library scale.

3.5 Mathematical Analogy in Automated Reasoning

Work on proof analogy in automated reasoning dates to the 1980s. Boy de la Tour and Caferra (1987) propose second-order pattern matching to transform known proof terms into candidates for analogous propositions. Melis and Veloso (1994) use derivational analogy at the level of proof plans to guide automated provers by replaying known analogous proofs. This thesis pursues analogy at a different level of abstraction—structural graph similarity across a large formal library—and aims at retrieval rather than direct proof transfer.

The cognitive science foundation for relational analogy is structure-mapping theory (Gentner, 1983; Falkenhainer et al., 1989), which holds that analogical reasoning aligns relational structure between domains rather than surface features. The multi-layer graph retrieval proposed in §5.3 can be viewed as a large-scale, data-driven realization of structure-mapping principles in a formal mathematical setting.

3.6 Theory Exploration and Conjecture Generation

Theory exploration systems automatically discover lemmas in a given formal theory development. Hipster (Johansson et al., 2014) integrates theory exploration into Isabelle/HOL using QuickSpec-style random testing to generate equational conjectures, then attempts to prove or prune them. LeanConjecturer (Onda et al., 2025) generates university-level mathematical conjectures for Lean 4 using LLMs, producing 12,289 conjectures from 40 Mathlib seed files.

Both are conceptually related to §5 (Missing Abstraction Detection) in that both attempt to identify mathematical statements "missing" from a library. This thesis's approach differs by using proof-state transition graphs and dependency topology to detect representation bottlenecks (repeated motifs, long coercion chains, high-branching bottleneck states) and evaluating suggestions by proof compression rather than by provability alone.

3.7 Library Learning and Abstraction Discovery

The program synthesis community has studied abstraction discovery as a library learning problem. DreamCoder (Ellis et al., 2021) jointly learns reusable library abstractions and a neural search policy through wake-sleep iteration. Stitch (Bowers et al., 2023) provides a significantly more efficient top-down abstraction algorithm using program compression ratio as the primary metric of abstraction quality. LAPS (Wong et al., 2021) extends DreamCoder with natural language annotations to guide library learning.

These systems are the program synthesis analogues of what this thesis proposes for formal mathematics. The key differences are: (i) the objects are types and propositions rather than input-output examples; (ii) correctness is guaranteed by Lean's type checker rather than test suites; (iii) the abstraction space includes typeclasses, coercions, and universal properties, which have no direct program synthesis analogue; and (iv) the corpus is a mature human-curated library rather than a set of synthetic tasks.

3.8 Perspectives on Mathematical Abstraction

Mitchell (2021) reviews AI approaches to abstraction and analogy-making, concluding that no current system achieves human-level analogical generalization and proposing evaluation criteria. Zhang et al. (2023) examine AI for mathematics from a cognitive science perspective, arguing that current systems lack the representation-change capabilities that make human mathematical reasoning powerful—a diagnosis this thesis takes as its starting point.

4. Core Hypotheses

This research program rests on five concrete hypotheses.

4.1 Hypothesis 1: Proof Traces Contain Recurring Cross-Domain Motifs

Lean proofs are not arbitrary tactic sequences. They contain recurring structural patterns: introducing witnesses, decomposing conjunctions, applying extensionality, transporting along equivalences, normalizing coercions, invoking universal properties, reducing goals by induction, or closing branches through simplification.

These motifs often recur across different mathematical domains. Prior empirical work on the Isabelle AFP (Blanchette et al., 2015) and manual pattern identification (Freitas and Whiteside, 2014) support the existence of such patterns; this thesis proposes to discover them automatically from proof-state transition graphs in Lean/mathlib.

4.2 Hypothesis 2: Theorem Analogy Can Be Retrieved Through Relational Structure

Many important mathematical analogies are not lexical. Two theorems may share little vocabulary but have similar dependency neighborhoods, type structures, proof skeletons, or relational patterns.

Structure-mapping theory (Gentner, 1983; Falkenhainer et al., 1989) predicts that the relevant similarity is relational, not featural. A retrieval system based on multi-layer formal graphs may find relational analogies that text-based systems such as LeanSearch (Gao et al., 2024) miss.

4.3 Hypothesis 3: Missing Abstractions Leave Measurable Traces

When a library lacks the right abstraction, users compensate through repeated proof patterns, duplicated lemmas, long coercion chains, brittle rewrite sequences, and similar theorem statements spread across namespaces. These phenomena should be detectable, turning abstraction discovery into a mathematical refactoring problem analogous to code clone detection (Roy and Cordy, 2007; Koschke, 2007).

4.4 Hypothesis 4: Abstraction Quality Can Be Measured by Compression

A good abstraction reduces the cost of future proof. Quantitative measures of abstraction value include: proof length reduction, dependency reuse, decrease in branching factor, reduction of repeated proof motifs, increased theorem transportability, and decrease in coercion and normalization burden. This compression-as-quality view is operationalized in program synthesis by Bowers et al. (2023); this thesis applies it in the richer setting of dependent type theory.

4.5 Hypothesis 5: Proof-State Topology Predicts Difficulty and Useful Intermediate Lemmas

A proof is a trajectory through a space of proof states. Some intermediate states act as bottlenecks: once reached, many downstream goals become easy. These bottleneck states may correspond to useful intermediate lemmas, invariants, or representation changes, and their geometry may guide lemma invention and proof planning.

4.6 Hypothesis 6: Proof Search Is Highly Non-Ergodic, and Good Representations Create Low-Dimensional Funnels

Proof search in a formal library is not uniform exploration of a search graph. It is highly non-ergodic: most tactic moves are useless, few intermediate states are reachable from a given goal, and a small number of lemmas act as bottlenecks through which many proof paths pass. Good mathematical representations do not merely simplify individual steps—they create low-dimensional proof funnels that collapse many equivalent search paths into one.

This suggests a deeper theoretical layer: the geometry of proof-state space can be characterized by notions analogous to bottleneck centrality, branching entropy, and proof-state diameter. Representation changes that reduce these quantities are the formal counterpart of mathematical insight.

5. Research Agenda

The agenda has four mutually reinforcing components.

5.1 Lean Graph Extraction and Normalization

The first task is to construct a graph-based representation of Lean/mathlib that is rich enough to capture mathematical structure but normalized enough to avoid irrelevant syntactic noise.

Existing tools provide a starting point. LeanDojo (Yang et al., 2023) extracts proof states, premise annotations, and tactic traces from Lean 4 repositories at scale. PACT (Han et al., 2021) demonstrates that rich training data can be extracted from Lean proof terms. This project extends such infrastructure to produce a multi-layer graph intermediate representation.

The system should extract multiple graph layers:

5.1.1 Declaration dependency graph

Nodes are definitions, theorems, instances, structures, and lemmas. Edges represent dependency, invocation, unfolding, instance use, or theorem application.

5.1.2 Expression graph

Theorem statements, definitions, and proof terms are represented as typed expression graphs capturing binders, applications, constants, equalities, propositions, functions, inductive types, and typeclass constraints.

5.1.3 Typeclass and coercion graph

This graph records how mathematical structures are connected through instances, coercions, inheritance, bundled structures, and canonical constructions.

5.1.4 Proof-state transition graph

For tactic proofs, each tactic transforms one or more goals into successor goals, inducing a transition graph over proof states. The graph records local context, target type, generated subgoals, tactic class, and closure mechanism.

5.1.5 Simplification and rewriting graph

`simp`, `rw`, normalization, and definitional equality hide substantial reasoning. The system exposes, where possible, the rewrite rules and normalization paths used to close goals.

The central design problem is normalization. Raw Lean expressions contain variable names, elaboration artifacts, syntactic choices, universe levels, implicit arguments, coercions, and proof-irrelevant details. The system must quotient out accidental differences while preserving mathematically meaningful structure.

Normalization strategy: neuro-symbolic, not purely symbolic. Lean expressions are notoriously difficult to normalize symbolically. Two mathematically identical proof states can have wildly different ASTs because they are *definitionally equal* (defeq), and raw terms contain universe variables, hygiene variables, invisible typeclass dictionaries, and elaboration artifacts that pure symbolic quotienting cannot easily handle. The chosen approach is therefore explicitly **neuro-symbolic**: symbolic preprocessing (alpha-renaming, implicit argument erasure, tactic categorization) reduces obvious syntactic variation, but the primary normalization mechanism is a learned graph embedding that maps syntactically diverse but mathematically equivalent structures to nearby points in a continuous latent space. Mathematical similarity then operates on this latent representation, absorbing the defeq noise that symbolic matchers choke on.

5.2 Proof Motif Mining

Once proof-state transition graphs are available, the next project is to discover recurring proof motifs.

A proof motif is a repeated local or global pattern in proof-state evolution. It is not merely a tactic sequence—it is a structural transformation pattern over goals, hypotheses, and generated subgoals.

The ML4PG family of systems (Komendantskaya et al., 2012; Heras and Komendantskaya, 2014) and empirical analysis of the Isabelle AFP (Blanchette et al., 2015) and manually curated proof patterns (Freitas and Whiteside, 2014) all provide prior evidence that recurring structures exist. The open question is whether they can be discovered automatically from proof-state transition graphs in a form useful for retrieval and proof planning, at the scale of Lean/mathlib.

The project mines motifs from learned graph embeddings rather than raw symbolic graphs. Exact frequent subgraph mining (FSM) is NP-hard and would computationally explode on mathlib's scale; symbolic matching also cannot handle the

definitional-equality ambiguity inherent in Lean's type theory. Instead, proof-state embeddings are computed by a GNN trained contrastively on the graph IR; motifs emerge as dense clusters in the resulting latent space, discoverable via approximate nearest-neighbor search and clustering (e.g., k-means or HDBSCAN). The result is a library of proof motifs with taxonomic structure.

Evaluation.

1. *Cross-domain recurrence*: Do motifs appear across multiple namespaces and mathematical areas?
2. *Predictive value*: Does motif identity help predict the next tactic or proof step?
3. *Retrieval value*: Can motif-aware retrieval find useful analogous proofs given a proof state?
4. *Compression value*: Can motifs summarize many proofs more compactly than tactic sequences?
5. *Human interpretability*: Do mathematicians or Lean users recognize mined motifs as meaningful strategies?

5.3 Theorem Analogy Retrieval

The next component is a retrieval system that finds structurally analogous theorems across mathlib. Given a theorem statement, proof state, or partial proof, the system returns other theorems similar not merely in text but in mathematical structure—motivated by structure-mapping theory (Gentner, 1983; Falkenhainer et al., 1989) and prior work on proof-plan analogy (Melis and Veloso, 1994).

Similarity is computed using a combination of theorem statement expression graphs, dependency neighborhoods, typeclass neighborhoods, proof motif signatures, proof-state transition summaries, namespace distance, shared abstraction usage, graph embeddings, and exact or approximate subgraph matching.

Evaluation.

1. *Known analogue pairs*: Human-curated pairs of analogous theorems as a held-out benchmark.
2. *Proof reuse prediction*: Whether retrieved theorems help prove held-out results.
3. *Namespace transfer*: Whether the system retrieves useful results from different mathematical domains.
4. *Ablation against baselines*: Compare BM25 over theorem names and docstrings; embedding-based search (Gao et al., 2024); symbol-overlap retrieval (Meng and Paulson, 2009); dependency-graph-only retrieval; proof-motif-only retrieval; and full multi-layer graph retrieval.

5. *Expert judgment*: Ask Lean users whether retrieved analogies are mathematically meaningful.

The key claim to test is: *formal graph structure contains analogy information not captured by theorem names or text embeddings alone.*

5.4 Missing Abstraction Detection

The most ambitious component is a system that detects where mathlib lacks the right representation.

This turns abstraction discovery into a mathematical refactoring problem (Roy and Cordy, 2007). Just as code clone detection identifies repeated code that should be factored into a shared abstraction, proof clone detection identifies repeated proof structure that signals a missing lemma, typeclass, or interface. The analogy with program synthesis library learning is also direct: both DreamCoder (Ellis et al., 2021) and Stitch (Bowers et al., 2023) identify reusable program substructures by finding patterns that improve compression; this thesis applies an analogous principle to formal proofs, exploiting the richer type-theoretic structure of Lean/mathlib.

Evaluation. Evaluation uses three complementary approaches.

Retrospective: Use mathlib history. Identify moments when important abstractions were introduced. Ask whether the system, using only the pre-abstraction state, would have detected the relevant cluster of representational pain.

Prospective: Run the system on current mathlib, produce suggestions, and have maintainers judge whether the suggestions are meaningful and whether accepted suggestions shorten proofs.

Synthetic: Remove or hide known lemmas and abstractions from a subset of the library and test whether the system rediscovers the need for them.

5.5 Abstraction Compression Metrics

Underlying all four components is the need for a quantitative theory of abstraction value. This component defines and empirically validates metrics for how much a mathematical abstraction compresses proof search.

Candidate metrics include: proof length reduction; dependency depth reduction; branching factor reduction; search entropy reduction; reuse centrality (how many downstream proofs invoke the abstraction); motif unification count (how many previously distinct motifs collapse into one); proof-state diameter reduction; transportability across domains; and **automated refactor compression**.

The last metric deserves special emphasis. When the system proposes a new intermediate lemma or abstraction, it automatically refactors downstream mathlib proof files to use it, recompiles, and measures the reduction in total proof size (AST depth, token count, tactic step count). A mathlib maintainer accepting such an

AI-generated structural refactor pull request—where downstream proofs become shorter, simpler, and cleaner— constitutes the most direct and undeniable empirical validation of the thesis. This should be the flagship evaluation for Project 3.

The key claim:

> A mathematical abstraction is valuable not merely because it is elegant, but because it changes the geometry of proof search.

5.6 Intermediate Lemma and Bottleneck Discovery

A focused application of proof-state geometry (Hypothesis 6) is to identify candidate intermediate lemmas directly from proof graphs. If many proof paths pass through similar intermediate states, those states correspond to reusable subgoals—natural candidates for new lemmas.

The contribution is a system that detects high-betweenness proof states and repeated subgoal shapes across different proofs, and proposes them as intermediate lemma candidates. Discovered lemmas can be directly supplied to proof search as premises, shortening proof traces and improving search coverage.

Evaluation. Can discovered lemmas shorten existing proofs? Can they help prove held-out theorems when added as premises? Do they correspond to lemmas that humans later added to mathlib? Do they improve tactic search success rate?

6. Technical Architecture

A plausible architecture has six layers.

6.1 Layer 1: Trace Collection

Instrument Lean to collect theorem statements, proof terms, tactic traces, proof states before and after tactics, generated subgoals, used lemmas, typeclass resolution traces, simplification traces, coercion insertions, and elaboration metadata. Existing infrastructure from LeanDojo (Yang et al., 2023) and PACT (Han et al., 2021) provides a foundation; the key extension is the proof-state transition graph and normalization pipeline.

6.2 Layer 2: Formal Graph IR

Convert raw Lean data into a normalized graph intermediate representation stable across superficial syntax changes. Work by Paliwal et al. (2020) and Bansal et al. (2019) on graph representations for HOL Light provides relevant prior art; the Lean 4 setting introduces additional structure through its typeclass and coercion system.

6.3 Layer 3: Motif and Pattern Mining

Mine repeated structures including frequent proof-state transitions, theorem statement schemas, recurring dependency neighborhoods, repeated coercion/rewrite paths, common proof skeletons, and bottleneck states. This layer combines symbolic graph algorithms with learned embeddings. The static dependency analysis of Huch (2022) on the Isabelle AFP demonstrates that large formal library graphs have measurable large-scale structure; this layer extends that analysis to proof-state dynamics.

6.4 Layer 4: Retrieval and Analogy

Build retrieval models over the graph IR using contrastive learning over multi-layer theorem graphs, with ablations against text-embedding baselines (Gao et al., 2024) and symbol-overlap baselines (Meng and Paulson, 2009). Additional methods to explore include graph kernels, subgraph matching, nearest-neighbor search over graph embeddings, and hybrid symbolic-neural retrieval.

6.5 Layer 5: Abstraction Scoring

Define metrics for representational inefficiency and abstraction value, including: repeated motif density, proof compression ratio (Bowers et al., 2023), dependency centrality, namespace-spanning recurrence, typeclass boundary friction, search entropy, proof-state branching factor, normalized proof length, and theorem transportability.

6.6 Layer 6: Interactive Assistant

Expose the system to users as an assistant for formal mathematics. Given a proof state, it can suggest analogous proof patterns, useful intermediate lemmas, repeated hypotheses, alternative transports, and potential missing abstractions. The final system is not merely an auto-prover; it is a representation-aware mathematical assistant.

7. Initial Projects

7.1 Project 1: Proof-State Motif Mining in Mathlib

Research question: Do formal proofs contain recurring structural motifs that are stable across domains and predictive of proof strategy?

Deliverables: proof-state graph extractor; motif mining algorithm; motif taxonomy; visualization tool; evaluation on tactic prediction and theorem retrieval.

Paper claim: Proof-state transition graphs contain reusable cross-domain motifs not captured by tactic sequences alone.

7.2 Project 2: Structural Theorem Analogy Retrieval

Research question: Can formal graph structure retrieve useful mathematical analogies beyond lexical or embedding-based theorem search?

Deliverables: multi-layer theorem graph representation; retrieval benchmark with human-curated analogue pairs; comparison against text-embedding baselines (Gao et al., 2024) and symbol-overlap baselines (Meng and Paulson, 2009); case studies of cross-domain analogies.

Paper claim: Multi-layer formal graph retrieval finds structurally analogous theorems across mathlib and improves proof reuse.

7.3 Project 3: Representation Inefficiency and Missing Lemma Discovery

Research question: Can missing abstractions be detected through repeated proof patterns and graph-theoretic bottlenecks?

Deliverables: representation inefficiency score; retrospective study using mathlib history; system-generated missing lemma candidates; evaluation through proof compression.

Paper claim: Formal libraries contain measurable representational pain points that can be used to suggest abstractions reducing proof complexity—detectable retrospectively from library history.

7.4 Project 4: Proof-State Geometry and Intermediate Lemma Discovery

Research question: Do high-betweenness proof states correspond to useful intermediate lemmas, and do library-mined bottleneck lemmas improve proof automation?

Deliverables: proof-state betweenness and entropy metrics; bottleneck state detector; candidate lemma generator; evaluation of whether suggested lemmas shorten held-out proofs or appear later in mathlib history.

Paper claim: Proof-state geometry reveals candidate intermediate lemmas that reduce proof complexity and improve downstream proof automation.

8. Why This Is Different From Existing AI Theorem Proving

Most current systems attempt to automate proof search, asking:

> Given a proof state, what tactic should be applied next?

This research asks a different question:

> Given a mathematical landscape, what representation would make proof search easier?

The contrast is precise. Current systems:

```
proof state → retrieve premises → generate tactic → search
```

This research:

```
formal library / proof traces → detect motifs, bottlenecks,
analogies, missing abstractions → reshape the search space
```

Tactic prediction is local; representation discovery is global. Tactic prediction imitates proof scripts; representation discovery studies why some proof scripts become short, reusable, and natural. Tactic prediction operates within the current library; representation discovery asks how the library itself should evolve. Tactic prediction tries to close goals; representation discovery tries to invent the objects that make goals close naturally.

This thesis also differs from the closest existing work on formal library analysis. Li et al. (2026) extract Mathlib's multilayer dependency graph and study its macroscopic structure but examine only static dependencies and do not pursue proof-state dynamics, motif retrieval, or abstraction detection. Huch (2022) applies the same approach to the Isabelle AFP with the same limitation. Blanchette et al. (2015) mine the AFP for descriptive proof statistics but do not use the results operationally. TacMiner (Xin et al., 2025) builds Tactic Dependence Graphs over Lean 4 proofs to discover reusable tactic libraries—the closest prior work to Project 1—but targets tactic compression rather than representation discovery. All four works use formal library structure descriptively or for tactic automation; this thesis uses it as an active substrate for representation discovery.

The long-term goal is not just an AI that proves existing theorems, but an AI that helps mathematicians build better mathematical languages.

9. Why Lean Is the Right Substrate

Lean is especially suitable for this research because it combines: a large and actively developed mathematical library (The mathlib Community, 2020); expressive dependent type theory; rich typeclass mechanisms; tactic-based proof development; machine-checkable proof terms; substantial use of abstraction and hierarchy; enough library history to study abstraction evolution; an active community of expert formalizers; and existing extraction infrastructure (Yang et al., 2023; Han et al., 2021).

The richness of the Lean typeclass system provides additional motivation. Baanen (2022) analyzes how mathlib uses Lean's instance parameter mechanism to organize its algebraic, order, topology, and analysis hierarchies, showing that representation choices in the typeclass system are non-trivial, consequential, and frequently interwoven in ways that create design tension—supporting the thesis's premise that Lean/mathlib contains substantial traces of representation decisions worth studying.

The HOL Light ecosystem provides a useful comparison point. HOList (Bansal et al., 2019) and the GNN work of Paliwal et al. (2020) demonstrate that graph-structured formal proof data supports meaningful machine learning. Lean/mathlib offers a richer typeclass hierarchy, a larger and more systematically organized library, and a

more active community.

Other proof assistants are also relevant. The Isabelle AFP has been analyzed graphically (Huch, 2022) and statistically (Blanchette et al., 2015), and the Mizar Mathematical Library has a long history as a formal corpus. These libraries may serve as secondary validation datasets to test whether mined motifs and detected abstractions generalize across systems.

10. Expected Contributions

To AI theorem proving. Representation-aware retrieval, proof planning, intermediate lemma discovery, and abstraction-guided search.

To formal methods. Tools for library refactoring, proof maintenance, theorem organization, and abstraction quality measurement.

To programming languages. A treatment of formal mathematics libraries as evolving typed knowledge systems, with abstraction studied as an empirical software phenomenon.

To machine learning. New graph-structured learning problems grounded in rigorous symbolic data.

To mathematics. Systems that may help mathematicians discover analogies, transport ideas across fields, and identify the right objects to define.

To the philosophy and cognitive science of mathematics. A computational way to study abstraction, analogy, representation, and proof compression—operationalizing theoretical frameworks such as structure-mapping theory (Gentner, 1983) on large-scale formal data (Mitchell, 2021; Zhang et al., 2023).

11. Long-Term Vision

The long-term vision is an AI-native environment for formal mathematics in which the system does not merely autocomplete proofs, but actively helps users navigate the space of mathematical representations.

A mathematician working in such an environment could ask:

- "What does this theorem resemble?"
- "Where has this proof pattern appeared before?"
- "What abstraction would unify these lemmas?"
- "What intermediate lemma would make this proof modular?"
- "Is this difficulty mathematical, or caused by library representation?"

- "Can this theorem be transported to another domain?"
- "What concept is missing from this part of the library?"

And the system would respond not just with tactic suggestions, but with structural observations:

- "This resembles a proof pattern from homological algebra."
- "You are missing an intermediate lemma of this shape."
- "This cluster of theorems suggests a new abstraction here."
- "Your proof is hard because the representation is wrong—transport through this equivalence."
- "Quotient these equivalent states first."
- "Define this invariant before proceeding."

The program synthesis community has shown that library learning from a corpus of programs is feasible: DreamCoder (Ellis et al., 2021) and Stitch (Bowers et al., 2023) demonstrate that useful abstractions can be discovered by mining reusable structure and measuring compression. This thesis pursues an analogous program for formal mathematics, where the corpus is Lean/mathlib, the abstractions are mathematical definitions and typeclasses, and correctness is guaranteed by the type checker rather than by test suites. The richer type-theoretic structure of formal mathematics creates both additional challenges (the abstraction space includes coercions, universal properties, and bundled structures with no direct program synthesis analogue) and additional opportunities (graph structure is richer, and correctness is mechanically verified).

In this environment, Lean becomes more than a proof checker. It becomes a microscope for mathematical structure.

The most ambitious version of the thesis is:

> Formal mathematics libraries are the first large-scale datasets from which machines can learn the dynamics of mathematical abstraction.

If this is true, then the future of AI for mathematics is not merely automated proof. It is automated representation discovery.

The path from the four initial projects to this vision requires demonstrating, sequentially: (i) that proof-state motifs exist and are cross-domain; (ii) that graph-based analogy retrieval outperforms text-based retrieval on a held-out benchmark; (iii) that the system's missing-abstraction suggestions correspond to genuine improvements in mathlib; and (iv) that proof-state geometry yields useful intermediate lemmas and quantifiable compression metrics. Each step is independently publishable and provides the empirical foundation for the next.

12. Risks and Failure Modes

Risk 1: Lean artifacts may overwhelm mathematical signal—especially definitional equality. Proof states contain many artifacts from elaboration, typeclass resolution, coercions, and naming choices. The deepest version of this problem is *definitional equality* (defeq): two mathematically identical proof states can have wildly different ASTs because Lean transparently unfolds definitions, inserts coercions, and resolves implicit arguments in varied ways. Hygiene macros also produce variables like `x✝` with no mathematical meaning. A purely symbolic normalization approach will spend years battling the Lean compiler before reaching any learning phase. *Mitigation*: adopt a neuro-symbolic strategy (§5.1): symbolic preprocessing handles obvious syntactic variation, while a GNN embedding absorbs defeq noise. Supplement with ablation studies, cross-namespace validation, and comparison across different formalizations.

Risk 2: Graph similarity may not correspond to meaningful analogy. Two theorem graphs may look similar for superficial reasons, while deep analogies may be structurally distant. *Mitigation*: combine symbolic graph features with expert judgment, proof reuse evaluation, and retrieval-based downstream tasks. Benchmark against human-curated analogue pairs.

Risk 3: Missing abstraction detection may produce vague suggestions. It is easy to identify repeated patterns; it is harder to propose a clean abstraction. *Mitigation*: begin with missing lemma discovery and proof compression before attempting new structure invention.

Risk 4: Full proof-state extraction may be technically expensive. Tracing all of mathlib may be large, brittle, or slow. LeanDojo (Yang et al., 2023) provides starting infrastructure but is not optimized for full proof-state transition graph extraction. *Mitigation*: start with selected domains, cache aggressively, support incremental extraction, and focus first on tactic-level traces and dependency graphs.

Risk 5: The project may become too philosophical. The big picture is attractive, but the field will only move if concrete systems and evaluations exist. *Mitigation*: anchor the research agenda in four initial artifacts (motif miner, analogy retriever, missing abstraction detector, bottleneck lemma discoverer), each with a concrete, reproducible evaluation. The automated PR to mathlib metric (§5.5) provides an external, undeniable benchmark that grounds the most ambitious claims.

Risk 6: Subgraph mining may not scale to mathlib. Exact frequent subgraph mining (FSM) is NP-hard. Applying it directly to a multi-layer graph with hundreds of thousands of nodes will computationally explode, and approximate symbolic FSM still struggles with the defeq ambiguity described in Risk 1. *Mitigation*: do not use exact symbolic FSM. Embed proof-state graphs with a GNN, then cluster the latent space with approximate nearest-neighbor methods (HDBSCAN, k-means). Motifs become dense clusters in embedding space rather than exact graph isomorphism

classes, reducing an NP-hard combinatorial search to a scalable geometric clustering problem.

13. Positioning Statement

This research direction is not only "AI for theorem proving," "graph mining on mathlib," or "retrieval for Lean." It is the study and automation of how formal mathematical representations are created, compressed, reused, and transported.

The best short slogans are:

- **From proof search to representation search.**
- **Mathematical abstraction is search-space compression.**
- **Lean as a microscope for the geometry of proof.**

A concise positioning statement:

> We use Lean/mathlib as a machine-readable record of mathematical abstraction. By extracting and analyzing proof-state graphs, theorem dependency graphs, and typeclass structures, we build systems that discover proof motifs, retrieve cross-domain analogies, detect missing abstractions, and guide representation-aware proof search.

Shorter: *this project turns formal mathematics libraries into datasets for learning mathematical representation discovery.*

14. First Paper Sketch

Possible title. *Proof Motifs in Formal Mathematics: Mining Reusable Proof-State Patterns from Lean/mathlib*

Core claim. Formal proof traces contain recurring structural motifs that appear across mathematical domains and improve theorem retrieval and proof-step prediction.

Contributions.

1. A proof-state graph extraction pipeline for Lean 4, extending LeanDojo (Yang et al., 2023) with transition-graph structure.
2. A normalized representation of tactic-induced proof-state transitions.
3. A motif mining algorithm for formal proofs.
4. An empirical taxonomy of recurring proof motifs in mathlib.
5. Evidence that motif features improve theorem retrieval or proof guidance.

Evaluation. Motif recurrence across namespaces; prediction of next tactic category; retrieval of proof-similar theorems; human evaluation of motif interpretability; ablation against tactic-sequence baselines, theorem-text baselines (Gao et al., 2024), and dependency-graph-only baselines (Huch, 2022).

Why this paper matters. It demonstrates that formal proof traces contain reusable higher-level structure beyond tactic sequences. This justifies the broader agenda of representation-aware AI for mathematics and distinguishes this work from tactic prediction systems (Lample et al., 2022; Yang et al., 2023) and prior static library analysis (Huch, 2022; Blanchette et al., 2015).

15. Final Thesis

The history of mathematics is partly the history of better representations. New objects, abstractions, and languages make previously impossible proofs natural. Until recently, this process was visible only through books, papers, lectures, and human interpretation. Formal mathematics changes that. It records mathematical representation in executable, machine-readable form.

Lean/mathlib (The mathlib Community, 2020) therefore gives us a new scientific object: a large-scale formal trace of mathematical abstraction in action.

The opportunity is to build systems that can read this trace—not merely to imitate proof steps, but to learn the structural patterns by which mathematics organizes itself.

> By treating formal mathematics libraries as graph-structured records of representation, proof, and abstraction, we can build AI systems that retrieve deep analogies, discover reusable proof motifs, identify missing abstractions, and guide mathematicians toward better representations. This shifts AI for mathematics from local proof search toward representation discovery—from searching in a fixed graph to searching for better graphs.

Mathematical abstraction is search-space compression. Lean/mathlib is the first dataset large enough to study this empirically.

This is the direction worth leading.

References

Anne Baanen. Use and abuse of instance parameters in the Lean mathematical library. In *Interactive Theorem Proving (ITP 2022)*, volume 237 of *Leibniz International Proceedings in Informatics*, pages 4:1–4:20. Schloss Dagstuhl, 2022. doi: 10.4230/LIPIcs.ITP.2022.4.

Kshitij Bansal, Sarah M. Loos, Markus N. Rabe, Christian Szegedy, and Stewart Wilcox. HOList: An environment for machine learning of higher-order theorem

proving. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 454–463, 2019.

Jasmin C. Blanchette, Cezary Kaliszyk, Lawrence C. Paulson, and Josef Urban. Hammering towards QED. *Journal of Formalized Reasoning*, 9(1):101–148, 2016.

Jasmin Christian Blanchette, Maximilian Haslbeck, Daniel Matichuk, and Tobias Nipkow. Mining the archive of formal proofs. In *Intelligent Computer Mathematics (CICM 2015)*, volume 9150 of *Lecture Notes in Computer Science*, pages 3–17. Springer, 2015. doi: 10.1007/978-3-319-20615-8_1.

Matthew Bowers, Theo X. Olausson, Lionel Wong, Gabriel Grand, Joshua B. Tenenbaum, Kevin Ellis, and Armando Solar-Lezama. Top-down synthesis for library learning. *Proceedings of the ACM on Programming Languages*, 7(POPL), 2023. doi: 10.1145/3571234.

T. Boy de la Tour and R. Caferra. Proof analogy in interactive theorem proving: A method to express and use it via second order pattern matching. In *Proceedings of the Sixth National Conference on Artificial Intelligence (AAAI-87)*, pages 95–99, 1987.

DeepSeek-AI. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025.

Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, PLDI 2021, pages 835–850. ACM, 2021. doi: 10.1145/3453483.3454080.

Brian Falkenhainer, Kenneth D. Forbus, and Dedre Gentner. The structure-mapping engine: Algorithm and examples. *Artificial Intelligence*, 41(1):1–63, 1989. doi: 10.1016/0004-3702(89)90077-5.

L. Freitas and I. Whiteside. Proof patterns for formal methods. In *FM 2014: Formal Methods*, volume 8442 of *Lecture Notes in Computer Science*, pages 279–294. Springer, 2014. doi: 10.1007/978-3-319-06410-9_20.

Guoxiong Gao, Haocheng Ju, Jiedong Jiang, Zihan Qin, and Bin Dong. A semantic search engine for Mathlib4, 2024.

Dedre Gentner. Structure-mapping: A theoretical framework for analogy. *Cognitive Science*, 7(2):155–170, 1983. doi: 10.1207/s15516709cog0702_3.

Google DeepMind AlphaProof Team. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 2025. doi: 10.1038/s41586-025-09833-y.

Jesse Michael Han, Jason Rabe, and Floris van Doorn. Proof artifact co-training for theorem proving with language models, 2021.

Jónathan Heras and Ekaterina Komendantskaya. Recycling proof patterns in Coq:

Case studies. *Mathematics in Computer Science*, 8(1):99–116, 2014. doi: 10.1007/s11786-014-0173-1.

Fabian Huch. Formal entity graphs as complex networks: Assessing centrality metrics of the archive of formal proofs. In *Intelligent Computer Mathematics (CICM 2022)*, volume 13467 of *Lecture Notes in Computer Science*, pages 148–163. Springer, 2022. doi: 10.1007/978-3-031-16681-5_10.

Moa Johansson, Dan Rosén, Nicholas Smallbone, and Koen Claessen. Hipster: Integrating theory exploration in a proof assistant. In *Intelligent Computer Mathematics (CICM 2014)*, volume 8543 of *Lecture Notes in Artificial Intelligence*, pages 108–122. Springer, 2014. doi: 10.1007/978-3-319-08434-3_9.

Cezary Kaliszyk and Josef Urban. Learning-assisted automated reasoning with Flyspeck. *Journal of Automated Reasoning*, 53(2):173–213, 2014. doi: 10.1007/s10817-014-9303-3.

Ekaterina Komendantskaya, Jónathan Heras, and Gudmund Grov. Machine learning in proof general: Interfacing interfaces, 2012.

Rainer Koschke. Survey of research on software clones. In *Dagstuhl Seminar Proceedings 06301: Duplication, Redundancy, and Similarity in Software*, 2007.

Daniel Kühlwein, Jasmin C. Blanchette, Cezary Kaliszyk, and Josef Urban. MaSh: Machine learning for Sledgehammer. In *Interactive Theorem Proving (ITP 2013)*, volume 7998 of *Lecture Notes in Computer Science*, pages 35–50. Springer, 2013. doi: 10.1007/978-3-642-39634-2_6.

Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. HyperTree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Xinze Li, Nanyun Peng, Simone Severini, and Patrick Shafto. The network structure of Mathlib, 2026.

Erica Melis and Manuela Veloso. Analogy makes proofs feasible. In *AAAI-94 Workshop on Case-Based Reasoning*, 1994.

Jia Meng and Lawrence C. Paulson. Lightweight relevance filtering for machine-generated resolution problems. *Journal of Applied Logic*, 7(1):41–57, 2009. doi: 10.1016/j.jal.2007.07.004.

Melanie Mitchell. Abstraction and analogy-making in artificial intelligence. *Annals of the New York Academy of Sciences*, 2021. doi: 10.1111/nyas.14619.

Naoto Onda, Kazumi Kasaura, Yuta Oriike, Masaya Taniguchi, Akiyoshi Sannai, and Sho Sonoda. LeanConjecturer: Automatic generation of mathematical conjectures for theorem proving, 2025.

Aditya Paliwal, Sarah M. Loos, Markus N. Rabe, Kshitij Bansal, and Christian Szegedy. Graph representations for higher-order logic and theorem proving. In

Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, pages 2967–2974, 2020.

Lawrence C. Paulson and Jasmin C. Blanchette. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *Proceedings of the 8th International Workshop on the Implementation of Logics (IWIL 2010)*, 2010.

Chanchal K. Roy and James R. Cordy. A survey on software clone detection research. Technical Report 541, Queen's University, 2007.

Amitayush Thakur, George Tsoukalas, Yeming Wen, Jimmy Xin, and Swarat Chaudhuri. An in-context learning agent for formal theorem-proving, 2023.

The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, CPP 2020, pages 367–381. ACM, 2020. doi: 10.1145/3372885.3373824.

Catherine Wong, Kevin Ellis, Joshua B. Tenenbaum, and Jacob Andreas. Leveraging language to learn program abstractions and search heuristics. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 11193–11204, 2021.

Yutong Xin, Jimmy Xin, Gabriel Poesia, Noah Goodman, Qiaochu Chen, and Isil Dillig. Automated discovery of tactic libraries for interactive theorem proving. *Proceedings of the ACM on Programming Languages*, 9(OOPSLA2), 2025. doi: 10.1145/3763121.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J. Prenger, and Anima Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Cedegao E. Zhang, Katherine M. Collins, Adrian Weller, and Joshua B. Tenenbaum. AI for mathematics: A cognitive science perspective, 2023.